

Xử lý & Làm sạch Dữ liệu

🔗 Bài này đánh giá bạn biết **SUY LUẬN** đằng sau mỗi quyết định xử lý — không phải chỉ biết gọi đúng hàm Pandas. Mỗi bước xử lý phải có Markdown giải thích lý do.

MỤC TIÊU GIAI ĐOẠN

🎯 Kết thúc Giai đoạn 2, bạn cần làm được:

Nhận một dataset thực tế bất kỳ có 5–10 lỗi khác nhau — phân tích được tất cả vấn đề, đưa ra quyết định xử lý có lý luận cho từng trường hợp, và viết được pipeline sạch, tái sử dụng được dưới dạng hàm Python.

🧠 Tư duy

- ▶ Biết đặt câu hỏi: dữ liệu thiếu này có ý nghĩa gì?
- ▶ Phân biệt null vô tình với null mang thông tin
- ▶ Giải thích quyết định bằng ngôn ngữ phi kỹ thuật

🔧 Kỹ thuật

- ▶ Xử lý missing, duplicate, outlier
- ▶ Convert dtype, chuẩn hóa chuỗi
- ▶ Normalize, encode categorical
- ▶ Viết pipeline hàm tái sử dụng

📄 Báo cáo

- ▶ Data Quality Report markdown
- ▶ Decision log: quyết định + lý do
- ▶ So sánh trước/sau: shape, null, dtype
- ▶ Hàm `clean_data()` có docstring

⚠ Chấm điểm Giai đoạn 2

Sinh viên thường biết gọi `df.fillna(0)` nhưng không biết tại sao điền 0 lại sai cho cột lương. Mọi bước xử lý trong bài này đều yêu cầu có cell Markdown giải thích. Thiếu explanation = không tính điểm phần đó.

TUẦN 1 — MISSING VALUES, DUPLICATES & DATA TYPES

1.1 Missing Values — không phải null nào cũng xử lý giống nhau

Trước khi xử lý, phải trả lời: null này xuất hiện vì lý do gì?

Trường hợp	❌ Sai / Thiếu suy luận	✅ Đúng — có lý do
Cột age: 30% null	<code>fillna(0)</code> → tuổi = 0 vô nghĩa	✓ <code>fillna(median)</code> vì age thường phân phối lệch phải; ghi rõ lý do
Cột salary: 60% null	<code>fillna(mean)</code> cho nhanh	✓ Xem xét drop cột hoặc MNAR — người không khai lương có thể khác nhau

Cột ghi chú: 70% null	dropna() toàn bộ hàng	✓ Null ở free-text = hợp lệ; giữ nguyên hoặc fillna('none')
Cột ngày sinh: 5% null	fillna(một ngày cố định)	✓ Xem xét interpolate hoặc drop hàng nếu ngày là key field

```
# Bước 1: Tổng quan missing — chạy đầu tiên
def missing_report(df):
    miss = df.isnull().sum()
    pct = (miss / len(df) * 100).round(2)
    return (pd.DataFrame({'count': miss, 'pct': pct})
            .query('count > 0').sort_values('pct', ascending=False))

# Bước 2: Quyết định có lý do — viết comment giải thích
# LÝ DO: age phân phối lệch phải => dùng median thay vì mean
df['age'] = df['age'].fillna(df['age'].median())

# LÝ DO: country null = không ảnh hưởng phân tích chính => giữ
df['country'] = df['country'].fillna('Unknown')
```

Checklist 1.1 — Missing Values:

- ☐ Chạy missing_report(), báo cáo số lượng và % missing từng cột
- ☐ Giải thích trong Markdown: cột nào dùng chiến lược gì và tại sao
- ☐ Áp dụng ít nhất 3 chiến lược khác nhau: drop, fillna, interpolate
- ☐ So sánh null count trước và sau xử lý trong 1 bảng
- ☐ Xác định loại null (MCAR/MAR/MNAR) cho ít nhất 2 cột — giải thích

1.2 Duplicates — kiểm tra xong phải hỏi tại sao xuất hiện

```
# Kiểm tra duplicate toàn hàng
n_dup = df.duplicated().sum()
print(f'Duplicate rows: {n_dup} ({n_dup/len(df)*100:.1f}%)')

# Kiểm tra duplicate theo key columns
key_dup = df.duplicated(subset=['email', 'timestamp']).sum()

# LÝ DO: duplicate toàn hàng = nhập liệu nhiều lần => giữ first
df = df.drop_duplicates(keep='first').reset_index(drop=True)
```

Checklist 1.2 — Duplicates:

- ☐ Kiểm tra duplicate toàn hàng và theo key columns
- ☐ Giải thích bằng Markdown: tại sao có duplicate? Lỗi nhập liệu, join sai, hay có ý nghĩa khác?
- ☐ Báo cáo số hàng trước và sau khi xử lý

1.3 Data Types — dtype sai là nguồn gốc của 40% lỗi phân tích

```
# Kiểm tra và convert dtype – ví dụ với Salary Survey

# Cột lương: object => float (xóa dấu phẩy và ký hiệu $)
df['annual_salary'] = (df['annual_salary']
    .str.replace(',','').str.replace('$','').astype(float))

# Cột ngày: object => datetime
df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')

# Cột category: object => category (tiết kiệm bộ nhớ)
df['industry'] = df['industry'].astype('category')

# Kiểm tra sau khi convert
df.dtypes # phải thấy float64, datetime64, category
```

Checklist 1.3 — Data Types:

- ☐ In df.dtypes trước xử lý, xác định các cột có dtype sai
- ☐ Convert cột số lưu dạng string (có dấu phẩy, ký hiệu tiền tệ) sang float
- ☐ Convert cột ngày/giờ sang datetime, xử lý lỗi bằng errors='coerce'
- ☐ Giải thích trong Markdown: tại sao dtype sai gây ra lỗi trong phân tích?
- ☐ Convert cột categorical sang dtype 'category' và đo tỉ lệ tiết kiệm bộ nhớ

2.1 Outliers — phát hiện đúng còn quan trọng hơn xử lý

💡 Quan điểm cần có

Outlier không phải luôn là lỗi. Một CEO có lương 10 triệu USD là outlier về số, nhưng là dữ liệu hợp lệ. Trước khi xử lý phải hỏi: đây là lỗi đo lường hay giá trị thực?

```
import numpy as np

# Phương pháp 1: IQR — tốt cho dữ liệu lệch phải
def detect_outliers_iqr(df, col):
    Q1, Q3 = df[col].quantile([0.25, 0.75])
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
    return df[(df[col] < lower) | (df[col] > upper)]

# Phương pháp 2: Z-score — tốt cho phân phối chuẩn
def detect_outliers_zscore(df, col, thresh=3):
    z = (df[col] - df[col].mean()) / df[col].std()
    return df[z.abs() > thresh]

# So sánh kết quả và giải thích sự khác biệt
```

Outlier là lỗi đo lường	Outlier hợp lệ nhưng hiếm	Không chắc chắn
▶ Xóa hoặc sửa giá trị ▶ Ví dụ: tuổi = -5 hoặc 200	▶ Giữ nguyên, ghi chú ▶ Ví dụ: CEO lương rất cao	▶ Winsorize hoặc cap ▶ Ghi rõ giả định

Checklist 2.1 — Outliers:

- ☐ Phát hiện outlier bằng IQR cho ít nhất 2 cột numeric, vẽ boxplot minh họa
- ☐ Phát hiện outlier bằng Z-score, so sánh kết quả với IQR
- ☐ Với mỗi outlier tìm được: đây là lỗi hay giá trị hợp lệ? Tại sao?
- ☐ Chọn và áp dụng 1 chiến lược xử lý — giải thích trong Markdown
- ☐ Thử winsorize (clip tại 1st/99th percentile) và so sánh với drop

2.2 Chuẩn hóa chuỗi — kỹ năng thực tế quan trọng nhất

Free-text field như cột `job_title` có thể có 50+ cách viết khác nhau cho cùng 1 nghề. Chuẩn hóa là kỹ năng tách biệt người phân tích giỏi.

```
# Pipeline chuẩn hóa chuỗi — thứ tự quan trọng
df['job_title'] = (df['job_title']
                  .str.lower()           # bước 1: lowercase trước
                  .str.strip()          # bước 2: bỏ khoảng trắng
                  .str.replace(r'\s+', ' ', regex=True) # khoảng trắng đơn
```

```

        .str.replace(r'^a-z0-9 ', '', regex=True) # bỏ ký tự đặc biệt
    )

# Gom nhóm các giá trị tương đồng bằng mapping dict
title_map = {
    'sw engineer': 'software engineer',
    'swe': 'software engineer',
    'software dev': 'software engineer',
}
df['job_title'] = df['job_title'].replace(title_map)

# Kiểm tra kết quả
df['job_title'].value_counts().head(10)

```

Checklist 2.2 — Chuẩn hóa chuỗi:

- ☐ Áp dụng pipeline: lower → strip → replace whitespace cho tất cả cột text
- ☐ Tìm và gom nhóm ít nhất 5 giá trị tương đồng bằng mapping dict
- ☐ Giải thích trong Markdown: tại sao phải chuẩn hóa trước khi phân tích?
- ☐ Dùng thư viện fuzzywuzzy hoặc rapidfuzz để tìm các giá trị gần giống nhau tự động

2.3 Normalization — khi nào dùng, khi nào không cần

Min-Max Normalization

Scale về [0, 1]. Dùng khi:

- ▶ Khoảng giá trị có ý nghĩa (pixel, điểm thi)
- ▶ Không biết phân phối dữ liệu
- ▶ Dùng với mạng neural hoặc KNN

Standard Scaler (Z-score)

Mean=0, Std=1. Dùng khi:

- ▶ Dữ liệu gần phân phối chuẩn
- ▶ Có outlier (robust hơn Min-Max)
- ▶ Dùng với regression, SVM, PCA

```

from sklearn.preprocessing import MinMaxScaler, StandardScaler

num_cols = ['age', 'annual_salary', 'years_exp']

# LÝ DO chọn Min-Max: các cột này không phân phối chuẩn
# và muốn giữ khoảng [0, 1] để dễ so sánh
scaler = MinMaxScaler()
df[num_cols] = scaler.fit_transform(df[num_cols])

```

Checklist 2.3 — Normalization:

- ☐ Áp dụng Min-Max cho ít nhất 1 cột numeric, giải thích tại sao chọn phương pháp này
- ☐ Áp dụng Standard Scaler cho ít nhất 1 cột khác, so sánh kết quả với Min-Max
- ☐ Giải thích trong Markdown: khi nào KHÔNG cần normalize?
- ☐ Thử RobustScaler và giải thích khi nào nó tốt hơn StandardScaler

3.1 Encoding Categorical Variables — chọn đúng kỹ thuật

Loại biến	Phương pháp phù hợp	Ví dụ & Lý do
Nominal (không có thứ tự)	One-Hot Encoding	country, gender — không có quan hệ thứ bậc
Ordinal (có thứ tự)	Label / Ordinal Encoding	trình độ học vấn: HS < ĐH < ThS < TS
Nhiều giá trị (high cardinality)	Target Encoding / Hashing	tên thành phố (500+ giá trị) — OHE tạo 500 cột
Binary (2 giá trị)	Map {0, 1}	yes/no, true/false — OHE không cần thiết

```
# One-Hot Encoding — cho biến nominal
df = pd.get_dummies(df, columns=['country', 'gender'], drop_first=True)
# LÝ DO drop_first=True: tránh multicollinearity (dummy variable trap)

# Ordinal Encoding — cho biến có thứ tự
edu_map = {'High School': 1, 'Bachelor': 2, 'Master': 3, 'PhD': 4}
df['education'] = df['education'].map(edu_map)
# LÝ DO: thứ tự có ý nghĩa => preserve ordinal relationship

# Binary encoding
df['is_remote'] = df['is_remote'].map({'Yes': 1, 'No': 0})
```

Checklist 3.1 — Encoding:

- ☐ Phân loại tất cả cột categorical: nominal / ordinal / binary / high-cardinality
- ☐ Áp dụng ít nhất 2 kỹ thuật encoding khác nhau cho các cột phù hợp
- ☐ Giải thích trong Markdown: tại sao OHE trên cột có 100 giá trị unique là vấn đề?
- ☐ Thử Target Encoding cho 1 cột high-cardinality, so sánh với OHE

3.2 Feature Engineering — tạo biến mới có ý nghĩa

```
# Ví dụ với Salary Survey dataset

# Feature 1: Lương theo một năm kinh nghiệm
df['salary_per_exp'] = df['annual_salary'] / (df['years_exp'] + 1)
# LÝ DO: so sánh mức lương công bằng hơn giữa người mới và lâu năm

# Feature 2: Phân nhóm tuổi
df['age_group'] = pd.cut(df['age'],
    bins=[0, 25, 35, 45, 60, 100],
    labels=['<25', '25-35', '35-45', '45-60', '60+'])
# LÝ DO: phân tích pattern theo nhóm tuổi rõ ràng hơn continuous

# Feature 3: Tính năm từ timestamp
df['survey_year'] = df['timestamp'].dt.year
```

Checklist 3.2 — Feature Engineering:

- ☐ Tạo ít nhất 2 feature mới có ý nghĩa từ các cột hiện có
- ☐ Mỗi feature mới phải có Markdown giải thích: tạo ra để trả lời câu hỏi gì?
- ☐ Tạo feature từ cột datetime: hour, day_of_week, is_weekend, month

3.3 Viết Pipeline hoàn chỉnh — hàm tái sử dụng được

🔗 Yêu cầu cuối cùng

Tất cả các bước xử lý phải được đóng gói vào hàm `clean_data(df)` có docstring rõ ràng. Hàm này phải chạy lại được trên dữ liệu mới mà không cần sửa code.

```
def clean_data(df):  
    """  
    Pipeline làm sạch Salary Survey dataset.  
  
    Parameters: df (DataFrame) — raw data  
    Returns:    df_clean (DataFrame) — đã xử lý  
  
    Các bước xử lý:  
    1. Xóa duplicate  
    2. Convert dtype (salary, timestamp)  
    3. Xử lý missing values theo chiến lược từng cột  
    4. Chuẩn hóa chuỗi: job_title, industry  
    5. Xử lý outlier cột salary  
    6. Encode categorical columns  
    7. Tạo feature mới  
    """  
    df = df.copy() # không sửa in-place  
  
    # Bước 1: Duplicate  
    df = df.drop_duplicates().reset_index(drop=True)  
  
    # Bước 2: Convert dtype  
    df['annual_salary'] = pd.to_numeric(  
        df['annual_salary'].str.replace(',', ''), errors='coerce')  
    df['timestamp'] = pd.to_datetime(df['timestamp'], errors='coerce')  
  
    # Bước 3: Missing values (xem quyết định trong Data Quality Report)  
    df['age'].fillna(df['age'].median(), inplace=True)  
    df['country'].fillna('Unknown', inplace=True)  
  
    # Bước 4-7: ... (các bước còn lại)  
  
    return df  
  
df_clean = clean_data(df_raw)
```

Checklist 3.3 — Pipeline:

- ☐ Viết hàm `clean_data(df)` bao gồm tất cả bước đã làm, có docstring

- ☐ Hàm trả về df mới (không sửa in-place), chạy lại được trên df gốc
- ☐ Viết comment LÝ DO trong từng bước của pipeline
- ☐ Viết unit test kiểm tra pipeline: assert shape, dtype, null count sau khi chạy

Tại sao Report quan trọng hơn Code?

Code chỉ cho thấy bạn làm được gì. Report cho thấy bạn suy nghĩ được gì. Một pipeline xử lý giống nhau nhưng lý do khác nhau sẽ dẫn đến kết quả phân tích hoàn toàn khác nhau.

Cấu trúc Data Quality Report (viết trong Markdown cell cuối notebook):

1

Tổng quan dataset gốc

Shape, dtypes, tổng null, tổng duplicate. 1 bảng so sánh trước/sau cho tất cả chỉ số.

2

Vấn đề phát hiện

Liệt kê từng vấn đề: tên vấn đề, cột bị ảnh hưởng, mức độ (critical/minor), ví dụ cụ thể.

3

Quyết định xử lý

Với mỗi vấn đề: quyết định gì, có thể làm cách khác không, rủi ro nếu không xử lý.

4

Quyết định khó (ambiguous)

1 trường hợp khó suy luận nhất: trình bày 2 phương án, chọn 1 và giải thích tại sao.

5

Hạn chế còn lại

Sau xử lý, dữ liệu vẫn còn vấn đề gì? Ảnh hưởng thế nào đến kết quả phân tích?

Mẫu Decision Log (viết trong notebook cho mỗi bước):

```
## [Tên vấn đề]: Missing values trong cột annual_salary

**Quan sát**: 12% giá trị null (3.280/27.500 hàng)

**Phân tích**: người dùng không điền lương có thể vì:
- MNAR: người thu nhập thấp ngại khai
- MCAR: bỏ qua ngẫu nhiên

**Phương án 1**: Drop các hàng null
- Ưu: đơn giản, dữ liệu sạch
- Nhược: mất 12% data, có thể gây bias nếu MNAR

**Phương án 2**: Fillna bằng median theo industry
- Ưu: giữ được data, ước tính có điều kiện
- Nhược: nếu MNAR => bất đồng nhất bias

**Quyết định**: Chọn Phương án 2 + thêm cột flag 'was_imputed'
**Lý do**: Phân tích chính là so sánh lương theo ngành;
mất 12% có thể làm lệch ngành nào có lương thấp.
```

 Nộp qua GitHub repo

Tất cả file nộp qua GitHub. Commit cuối trước deadline được chấm. Ít nhất 4 commits khác nhau — không push 1 lần duy nhất.

#	File cần nộp	Format
01	salary_cleaning.ipynb — notebook xử lý đầy đủ, tất cả cell đã chạy, có Markdown explanation sau mỗi bước	.ipynb
02	data_quality_report.md — báo cáo 5 phần theo cấu trúc trên: tổng quan, vấn đề, quyết định, khó, hạn chế	.md
03	salary_cleaned.csv — dataset sau khi xử lý, export từ hàm clean_data()	.csv
04	README.md — mô tả repo, cách chạy code, tóm tắt các vấn đề chính đã xử lý	.md

RUBRIC CHẤM ĐIỂM

Tiêu chí	Điểm	Đạt điểm tối đa khi...
Kỹ thuật xử lý	3đ	Pipeline chạy được, xử lý đủ missing, duplicate, outlier, dtype, encoding. Hàm clean_data() tái sử dụng được.
Suy luận & lý do	3đ	Mỗi bước có Markdown giải thích rõ lý do. Decision log cho từng vấn đề. Quyết định khó được phân tích 2 phương án.
Data Quality Report	2đ	Đủ 5 phần, có bảng so sánh trước/sau, nêu hạn chế còn lại. Viết rõ ràng, không chung chung.
Feature Engineering	1đ	Ít nhất 2 feature mới có ý nghĩa, mỗi feature có giải thích tạo ra để trả lời câu hỏi gì.
Git & quy trình	1đ	Ít nhất 4 commits rõ ràng, repo đúng cấu trúc, README đầy đủ.
BONUS	+2đ	Unit test pipeline · fuzzywuzzy matching · RobustScaler · Target Encoding với giải thích chi tiết

 Mất điểm nghiêm trọng

Code có nhưng không có Markdown explanation (−1đ mỗi phần) · Cell lỗi không chạy được (−1đ) · Report chung chung, không có lý do cụ thể (−1đ) · Push 1 commit duy nhất vào phút chót (−0.5đ)

GỢI Ý LỊCH TRÌNH 3 TUẦN

Tuần	Trên lớp (3 giờ)	Tự thực hành (6 giờ)
Tuần 1	Missing values, Duplicates, Data Types — live-coding cùng dataset Salary Survey	Khám phá dataset, chạy missing_report(), xử lý các vấn đề cơ bản, viết phần 1–2 của Report
Tuần 2	Outliers, Chuẩn hóa chuỗi, Normalization — live-coding + peer	Phát hiện outlier cả 2 phương pháp, chuẩn hóa chuỗi, áp dụng scaler, viết phần 3–4 của Report

	review giữa các nhóm	
Tuần 3	Encoding, Feature Engineering, code review & thảo luận quyết định khó	Hoàn thiện pipeline hàm, viết phần 5 Report, chuẩn bị trình bày 5 phút trước lớp

Lời khuyên

Bài này không có đáp án duy nhất. Một sinh viên fillna(median) và một sinh viên drop null đều có thể đạt điểm tối đa nếu cả hai giải thích được lý do và hiểu rủi ro của quyết định mình chọn.